Lecture 25.04: Classification Evaluation Metrics

📌 Topic Overview

In this lecture, Shraddha ma'am covers the mathematical formulas and operational tradeoffs between **Accuracy**, **Precision**, **Recall (Sensitivity)**, and **F1 Score**, explaining when to prioritize Precision versus Recall.

📐 Mathematical Formulas

1. Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- Measures overall proportion of correct classifications.
- **Flaw**: Misleading when working with imbalanced datasets (e.g., predicting 99% non-fraud by simply predicting 0 every time).

2. Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

- Question answered: *"When the model predicts Positive, how often is it actually right?"*
- **Prioritize Precision when False Positives (FP) are costly** (e.g., Spam detection, loan approval).

3. Recall (Sensitivity)

$$\text{Recall} = \frac{TP}{TP + FN}$$

- Question answered: *"Out of all actual Positive cases, how many did the model successfully catch?"*

- **Prioritize Recall when False Negatives (FN) are dangerous** (e.g., Cancer detection, COVID screening, fire alarm).

4. F1 Score

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- Harmonic mean balancing Precision and Recall. Unlike arithmetic mean, harmonic mean severely punishes extreme values.

⚖️ Precision vs Recall Tradeoff



💻 Python Implementation

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_scor

def evaluate_classification_performance(y_true, y_pred):
    """
    Computes all standard classification metrics:
    - Accuracy:  Overall correctness
    - Precision: Confidence when predicting positive (Minimizes FP)
    - Recall:    Ability to capture all actual positives (Minimizes FN)
    - F1-Score:  Harmonic mean of Precision and Recall
    """
    acc  = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, zero_division=0)
    rec  = recall_score(y_true, y_pred, zero_division=0)
    f1   = f1_score(y_true, y_pred, zero_division=0)

    results = {
        'accuracy':  float(acc),
        'precision': float(prec),
        'recall':    float(rec),
        'f1_score':  float(f1)
```

```
    }
    return results

# --- Function Call & Testing ---
# Real-world scenario: Imbalanced medical test
y_actual    = [0, 0, 0, 0, 0, 0, 0, 1, 1, 1]
y_predicted = [0, 0, 0, 0, 0, 1, 0, 0, 1, 1]

metrics = evaluate_classification_performance(y_actual, y_predicted)
print(f"Accuracy:  {metrics['accuracy']:.3f}")
print(f"Precision: {metrics['precision']:.3f} (When model says Positive, it is rig
print(f"Recall:    {metrics['recall']:.3f} (Model caught 66.7% of actual disease c
print(f"F1-Score:  {metrics['f1_score']:.3f}")
```

🔑 Key Takeaways

1. Never rely on Accuracy alone on imbalanced datasets.
2. High Precision avoids false alarms; High Recall avoids missed detections.
3. F1-Score provides a single balanced metric when precision and recall must both be healthy.